

PRACTICAL WEEK 1

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- Write a C code to create a process using fork() system call.
- Read the user input as shell command.
- Use a system call exec() to execute the shell command.
- Use a wait() system call for waiting a parent process.
- Display the PIDs of parent and child processes.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6
7  int main() {
8      char command[100];
9      pid_t pid;
10
11     // Read Linux command from user
12     printf("Enter a Linux command: ");
13     fgets(command, sizeof(command), stdin);
14
15     // Create child process
16     pid = fork();
17
18     if (pid < 0) {
19         // fork() failed
20         perror("fork failed");
21         return 1;
22     }
23
24     if (pid == 0) {
25         // Child process
26         printf("\nChild Process\n");
27         printf("Child PID: %d\n", getpid());
28         printf("Parent PID: %d\n", getppid());
29
30         printf("\nExecuting command: %s\n", command);
31
32         // Execute command using shell
33         execl("/bin/sh", "sh", "-c", command, NULL);
34
35         // Executes only if execl() fails
36         perror("exec failed");
37         exit(1);
38     }
39     else {
40         // Parent process
41         printf("\nParent Process\n");
42         printf("Parent PID: %d\n", getpid());
43         printf("Child PID: %d\n", pid);
44
45         // Wait for child process
46         wait(NULL);
47
48         printf("\nChild process execution completed.\n");
49     }
50
51     return 0;
52 }

```

Output

```
Last login: Wed Aug 26 12:10:27 on ttys000
nrk@nallas-mac-Book-Air ~ % nano 3.c

nrk@nallas-mac-Book-Air ~ % gcc 3.c -o 3
nrk@nallas-mac-Book-Air ~ % ./ 3
zsh: permission denied: ./
nrk@nallas-mac-Book-Air ~ %
```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
nm PICO 5.89                                     File: 2.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int choice;

    do {
        printf("\n=====
        printf("HARDWARE AND OS RESOURCE MONITOR\n");
        printf("=====
        printf("\n");

        printf("1. Display Operating System Information (uname)\n");
        printf("2. Display CPU Information (lscpu)\n");
        printf("3. Display Running Processes (ps)\n");
        printf("4. Display System Resource Usage (top)\n");
        printf("5. Run All Commands\n");
        printf("6. Exit\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("\n--- OPERATING SYSTEM INFORMATION ---\n");
                system("uname -a");
                break;

            case 2:
                printf("\n--- CPU INFORMATION ---\n");
                system("lscpu");
                break;

            case 3:
                printf("\n--- RUNNING PROCESSES ---\n");
                system("ps -ef");
                break;

            case 4:
                printf("\n--- SYSTEM RESOURCE USAGE ---\n");
                printf("Press 'q' to exit TOP.\n");
                system("top");
                break;

            case 5:
                printf("\n--- OPERATING SYSTEM INFORMATION ---\n");
                system("uname -a");

                printf("\n--- CPU INFORMATION ---\n");
                system("lscpu");

                printf("\n--- RUNNING PROCESSES ---\n");
                system("ps -ef");

                printf("\n--- SYSTEM RESOURCE USAGE ---\n");
                printf("Press 'q' to exit TOP.\n");
                system("top");
                break;
        }
    } while(choice != 6);
}
```

```
Last login: Wed Aug 26 12:08:04 on ttys000
nrk@nallas-mac-Book-Air ~ % nano 2.c
```

```
nrk@nallas-mac-Book-Air ~ % gcc 2.c -o 2
nrk@nallas-mac-Book-Air ~ % ./2
```

```
=====
HARDWARE AND OS RESOURCE MONITOR
=====
1. Display Operating System Information (uname)
2. Display CPU Information (lscpu)
3. Display Running Processes (ps)
4. Display System Resource Usage (top)
5. Run All Commands
6. Exit
Enter your choice: █
```

Hardware Resource	OS Abstraction	OS Service
CPU	Processes/threads	CPU scheduling
Main memory	Virtual memory/address spaces	Memory management
Storage	Files/directories	File system
Keyboard/mouse	Input streams/devices	I/O management
Display	Device interface	Device management
Network adapter	Network interface	Network management



